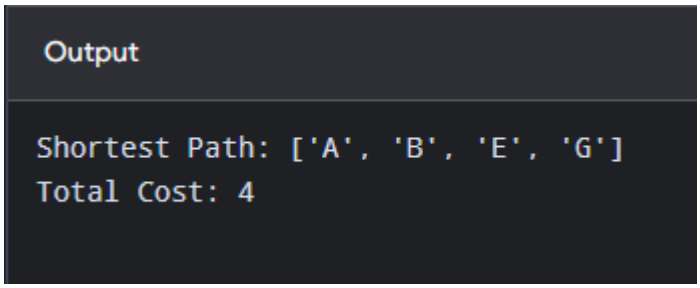
Lab experiment

A* Algorithm

Coding:

```
# A* Search Algorithm

import heapq

# Graph with edge costs
graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 3), ('E', 1)],
    'C': [('F', 5)],
    'D': [],
    'E': [('G', 2)],
    'F': [('G', 2)],
    'G': []
}

# Heuristic values
h = {
    'A': 5,
    'B': 4,
    'C': 4,
    'D': 3,
    'E': 2,
    'F': 1,
    'G': 0
}

# A* function
def astar(start, goal):
    queue = [(h[start], 0, start)]
    parent = {start: None}
    cost = {start: 0}

    while queue:
        f, g, node = heapq.heappop(queue)

        if node == goal:
            path = []
```

```
            while node is not None:
                path.append(node)
                node = parent[node]

            path.reverse()

            print("Shortest Path:", path)
            print("Total Cost:", cost[goal])
            return

        for neighbor, weight in graph[node]:
            new_cost = g + weight

            if neighbor not in cost or new_cost < cost[neighbor]:
                cost[neighbor] = new_cost
                parent[neighbor] = node

                f = new_cost + h[neighbor]

                heapq.heappush(
                    queue, (f, new_cost, neighbor)
                )

    print("No path found")

# Main program
astar('A', 'G')
```

Output: